Relational Reasoning, Memory, and Embedded Knowledge

Jeremy H. Carroll

Task-relative recovery and bounded recursive improvement

2026-10-23

Version 1.2.3

Abstract

Agents, records, instruments and evaluators are parts of the world they describe. Within an explicitly declared model, we characterize what a record determines, which tasks a memory can recover, and which distinctions an updater must preserve for future use. Exact recovery is equivalent to constancy on observation fibers when representatives are supplied. Future equivalence is the greatest update-stable relation preserving the declared readout. Constructive Lean proofs establish factivity, positive introspection, knowledge of one's own record, and preservation under every finite action sequence. Executable checked append and recall operations reject conflicting data, and a guarded transition preserves a declared invariant through arbitrary finite executions. These positive results accompany the earlier finite-carrier obstruction: indefinitely many distinct improvements cannot be represented by a fixed finite carrier. That result does not limit expanding histories or all forms of learning. Exact finite experiments exhibit a memory enlargement from one to two bits when a new action is admitted, a sharp recovery-error witness, and the advantage of adaptive observation. The companion paper supplies quantitative recovery, experiment-simulation and query-budget formalizations. Every result identifies its observation model, task and resource assumptions; no viewpoint outside the world is required. A source-checked persistent memory now enters the repository agent context before dispatch. Separate-process live trials demonstrate exact reuse of a learned observation and retain an arithmetic failure after correct recall. These finite results distinguish reliable memory access from reliable reasoning.

1. The world-internal starting point

The world is what it is. There is no outside or observer from nowhere. This is the organizing methodological rule. A state variable models part of the world; a record models something stored within it; an instrument and an evaluator are further interacting parts. A mathematical quantifier over candidate states does not posit a physically detached observer. The adequacy of the candidate model remains an empirical question.

We distinguish truth at a state, determination by a record, an effective procedure for computing an answer, and permission to act. The legacy code's `authorized` output is a policy value. It is not a definition of knowledge or truth. Different names for generator and verifier do not establish statistical independence or an accurate observation model.

Let a record map and a task be given by

$$r : X \to O, \qquad f : X \to Y.$$

The compatibility fiber contains the model states agreeing with the actual record. A proposition is determined by the record when it holds throughout that fiber:

$$C_r(x) = \{y : r(y) = r(x)\}, \qquad K_r p(x) \iff \forall y \in C_r(x),\ p(y).$$

This semantics does not by itself describe an agent's awareness, confidence, algorithm or physical ability to enumerate the fiber. False or omitted world models can make an otherwise valid calculation inapplicable. Observation noise requires an enlarged compatibility relation or a stated probabilistic model.

1.1 Constructive self-knowledge

Theorem 1 (Lean). Record-relative determination is factive, positively introspective, and includes the record's own value:

$$K_r p(x) \Rightarrow p(x), \qquad K_r p(x) \Rightarrow K_r(K_r p)(x), \qquad K_r(r = r(x))(x).$$

Factivity uses the actual state in its own fiber. Positive introspection uses transitivity of record equality. Own-record determination uses the definition of the fiber. `determined_is_true`, `positive_introspection` and `own_record_is_determined` are constructive proofs with no axiom dependencies. A more discriminating record preserves previously determined propositions (`refinement_preserves_determination`). These statements refute any reading of self-reference as a blanket impossibility of self-knowledge.

An anti-announced-bit task is different. No Boolean equals its own negation; an adversary defined to negate the announced answer prevents universal correctness for that protocol. `anti_prediction_not_universal` states exactly this obstruction. It says nothing against remembering one's previous outputs or correctly predicting a restricted transition system. Wolpert's inference-device restrictions require their setup-distinguishability hypotheses [1]. Logical induction and reflective-oracle constructions give qualified positive self-reference results under different formal models [2,3].

2. Recovery depends on the task

Theorem 2 (Lean). A decoder recovering the task from the record makes the task constant on every record fiber. Conversely, supplying a representative of each realized record constructs a decoder whenever the task is constant on fibers:

$$\exists d : O \to Y \ \forall x,\ d(r(x)) = f(x) \quad \iff \quad \forall x, y,\ r(x) = r(y) \Rightarrow f(x) = f(y).$$

The reverse direction explicitly assumes a representative function whose record agrees with each realized record. Its values on unrealized records are immaterial. This avoids silently invoking a choice principle or an algorithm for arbitrary equivalence classes. `recoverable_iff_fiber_constant` proves the equivalence at that type. A witnessed collision with different task values excludes every exact decoder. Deterministic postprocessing cannot separate equal records. Finite records can nevertheless be sufficient for a finite task, and many states may share a record without losing any answer that task requests.

For finite scalar tasks, the optimal uniform error is half the largest within-fiber diameter. The midpoint of each fiber's extrema attains it. The exact code reports both the midpoint and an extremal collision witness; the companion's `attained_fiber_minimax` proves the real-valued certificate principle. The two values 112 and 143 under one record force error at least 31/2; midpoint 255/2 attains it. Separating their records reduces the error to zero. This is a restriction on recovery from that record, not a restriction on obtaining an additional observation.

3. What a memory transformation is

A memory transformation is a concrete change to stored representation: appending an observation, summarizing a transcript, replacing a table with a smaller one, or updating a model after an action. Its correctness is relative to what the memory must still answer. A transformation can discard distinctions irrelevant to a declared task and preserve every answer to that task. Adding another task or action can make the same discarded distinction relevant.

An experiment declares the model identity, readouts, permitted actions, decoder, updater version, comparison class, error allowance, source bindings and storage budget. A changed task or action family is a changed contract. A checksum binds bytes; it does not establish that those bytes describe the world correctly.

Let an action word apply the declared transitions in order. Two states are future-equivalent when every such word yields the same readout:

$$x \sim_F y \quad \iff \quad \forall w \in A^*,\ q(F_w(x)) = q(F_w(y)).$$

Theorem 3 (Lean). Future equivalence is an equivalence relation, preserves the current readout, is stable under each action, and contains every other stable readout-preserving relation. The proof uses the empty word for current agreement and a prefixed word for stability; induction proves maximality. `stable_relation_refines_future_eq` supplies the last direction. Thus the quotient by this relation is the coarsest exact memory for the specified future tasks, at the level of existence of representations. Computing that quotient requires additional effective or finite structure.

This deterministic construction is related to minimal predictive states in computational mechanics and predictive state representations [4,5]. Those literatures have their own stochastic, stationarity and almost-sure qualifications. Information bottleneck objectives instead trade information retention against a chosen relevance distribution [6]; they do not automatically preserve every future action-conditioned task.

3.1 A sufficient implementation contract

For a memory encoder, decoder and updater, the two commuting conditions are

$$d(m(x)) = q(x), \qquad m(F_a(x)) = U_a(m(x)).$$

Theorem 4 (Lean). These conditions imply agreement of every future readout whenever the initial memories agree (`implemented_memory_preserves_future`). They also imply that updating encoded memory along any finite word gives the encoding of the corresponding world state (`run_updates_commute`). This second statement establishes correspondence of actual updates, not only indistinguishability of equal memories.

The executable witness uses a pair of Boolean values, an encoder storing them in reversed order, a decoder reading the second stored component, and identity/swap actions. `pair_memory_commutes` proves the updater equation; `pair_memory_preserves_all_words` proves exact readout preservation after any finite word. Remembering only the first world bit fails after swap: two initially identical records then expose different second bits. The Python example follows the same finite definitions; finite execution checks accompany the Lean proofs, without asserting universal Python-to-Lean refinement.

3.2 Capacity and constructive refinement

If a finite system has a given number of future-equivalence classes, any exact memory with the stated updater must have at least that many distinguishable states. The companion proves this by injectivity on representatives (`future_classes_require_memory`). Fixed-length binary storage then needs enough bit strings; variable-length expected coding requires a separate probability and coding model.

The reference algorithm refines a finite partition until every action preserves every class. Independently searching pairs of states produces distinguishing action words. For four states with readout `(0,0,1,1)`, identity alone yields two classes; adding the permutation `(0,2,1,3)` yields four. One bit suffices for the first contract and two for the second. This is a constructive reason to enlarge memory, rather than to conclude that remembering is impossible. Exhaustive four-state checks compare partition refinement against pair-state search on 65,536 state-pair cases.

4. Enforced updates, recall and safety

`MemoryImplementation.lean` defines a finite archive of record/answer pairs as compatible when equal records have equal answers. Its executable `appendChecked` returns the concatenated archive only if the entire result is compatible. `accepted_append_compatible` proves that a successful result satisfies the condition; `accepted_append_retains` proves that every old entry remains. A conflicting append is refused, leaving the caller's immutable old value available.

The executable `recallChecked` requires a matching entry and agreement of all stored entries with that record. `accepted_recall_sound` proves both coverage and agreement for every returned answer. A changed archive

containing conflicting answers refuses a formerly acceptable recall. These checks concern the complete declared archive, not unstored states. The exact-rational `checked_extension` additionally supports a nonnegative scalar error allowance and independently replays its complete enlarged archive.

Theorem 5 (Lean, enforced transition invariant). Given a decidable invariant, a guarded transition executes a proposed next state only when it satisfies that invariant; otherwise it retains the current state. Starting from an invariant-satisfying state, every finite guarded execution preserves the invariant. `gated_run_preserves` proves this for arbitrary types. The concrete pair-state gate rejects an overwrite to `(true,true)`, with `concrete_gate_preserves_every_word` proving the declared invariant for every finite sequence of overwrite requests.

The guard is part of the Lean transition function, so the theorem applies to enforced transitions in that implementation. It does not infer a physical safety predicate from a provenance label. A runtime bypass, unchecked recall entry point, incomplete archive, inaccurate predicate or unsafe initial state falls outside its premise. The included Python functions are a finite reference experiment. This abstract theorem alone does not establish persistence, agent-request injection or actual model use; section 8 supplies separate finite evidence for those implementation questions.

5. What the earlier finite RSI theorem actually bounds

The original `RsiImpossible.lean` and `RelationalRsiNoGo.lean` retain their definitions and theorem statements. A duplicate-free list contained in a carrier of size N has length at most N. `OmegaRelationalRsi.rsi_impossible` rules out an infinite strictly improving trajectory whose transitive, irreflexive dominance is invariant under report equality in both arguments and whose report image lies in that fixed finite carrier. It derives distinct reports from those hypotheses using `lemma1c_goodhart_metric_collapse_on_collision`, then applies the finite-list bound. The underlying state space can be infinite. Neither a particular self-modification mechanism nor statistical independence, primitive difference or a sheaf premise occurs in this theorem's signature.

N reports allow at most N−1 report-grounded strict transitions. An arbitrary sequence of N+1 observations repeats by transition index N; this says nothing about the magnitude of a jump or elapsed time. Transcripts, verifier state, timestamps and rolling context must be included in the declared report interface whenever dominance uses them.

Structural `RelationallyAligned` transformations and the numeric-rank `RelationalAlignment` predicate are different definitions, not equivalent encodings. The latter explicitly uses an internal score below N. Relabeling scores can change that policy. The compiled `ReviewCountermodels` exhibit a structurally aligned successor and an infinite increasing trajectory with one constant report; the dominance is not report-grounded. This is a scope control consistent with the finite-report theorem.

`RelationalRSI.rsi_decouples_within_carrier_bound` proves a horizon for its defined increasing internal counter and bounded carrier alignment. Licensed lifts consume a declared finite budget. Coupled-carrier results use a fixed finite product carrier. The overlap and higher-cochain theorems concern declared compatibility equations; failure to glue those data is a different obstruction. None of these results makes every self-modifying process halt or proves that all useful improvements cease.

`PrefixBounds.spending_prefix_invariant` bounds cumulative natural-valued charges plus remaining budget by initial budget, assuming the budget inequality only at indices below a declared finite horizon L. `refinement_prefix_bound` and `lift_prefix_bound` give the corresponding finite-prefix bounds. `interleaved_lift_event_bound` counts actual charged lift events and permits uncharged intervening ticks. The older total strictly decreasing sequences are impossible infinite-descent hypotheses, not certificates of existing finite executions. The retained asynchronous and rewiring corollaries are cardinality statements on fixed report products; they do not formalize buffers, evolving incidence or terminal-reason distributions.

A finite alphabet does not bound the number of complete histories. A binary alphabet has 127 words of length at most six, and arbitrarily many finite words without a length cap. A resource bound must therefore cover total stored configurations, duration, precision, history length or an explicitly fixed quotient. Finite physical dimension alone supplies no unlimited-repetition bound. Repeated measurements, expanding

memory and new interventions change the model. Conversely, computation can improve an algorithm even while its evidence is unchanged; observational identifiability and computational difficulty are separate limits.

The earlier policy, quantum and campaign discussion is retained verbatim under `docs/pre_integration_*` as historical source. Its extrapolations are superseded by the explicit scopes here. Evaluator selection does not alone imply zero mutual information; self-report is not automatically a destructive quantum pinching; Bell/contextuality does not establish that reality has no facts. The companion's contextuality results concern specified jointly representable distributions. Quantum statistical comparison requires completely positive and, where used, ancillary-system conditions [7]. No new quantum recovery theorem is claimed here.

6. Learning and intelligence as resource-dependent capabilities

A useful capability description specifies the environment family, task, observations, actions, working and persistent memory, computation, and accepted error. Lower bounds then identify an intervention that can remove the obstruction: acquire a new observation, retain a distinction, change an action family, take another data pass, or improve computation. These are different changes, with different costs.

The finite query experiment demonstrates the distinction. Three binary queries on four worlds admit a worst-case adaptive depth of two, while every fixed identifying panel requires all three. The companion proves the bounded binary-query recurrence equivalent to existence of a valid plan within that depth and proves error preservation at checked leaves. The finite Python solver returns a plan or a witnessed unresolved search outcome and independently checks optimality. Destructive measurements, forbidden actions and elapsed physical time require additional models.

Fano bounds require a distribution and information assumptions [8]. Continual-learning memory lower bounds apply to specified classes, sample regimes and pass constraints; extra passes can change them [9,10]. Universal-intelligence quantities depend on their environment coding and prior [11,12]. Gödel machines make conditional claims relative to a coded objective and proof system [13]. None supplies an assumption-free ceiling on all intelligence or learning.

Statistical improvement claims also need valid evidence after adaptive selection. Confidence sequences and adaptive-data-analysis results address specified sampling and stopping models [14,15]. The inspected September 2026 revision of Statistical Gödel Machines accounts for every opened confirmation, including failed ones, with conditionally valid tests and predictable summable error allocations [16]. Non-improvement under an objective is different from unsafe behavior. These statistical theorems are imported literature, not Lean results in this package.

7. Mechanization and reproducibility

`EmbeddedLimits.lean` is copied byte for byte from the October 22 research dossier; its twenty named theorems use no axioms. `MemoryImplementation.lean` supplies the executable append, recall, updater and gate proofs. The original finite-carrier modules retain their historical definitions and statements. The generated named census covers every explicit theorem in these modules, and the full census enumerates package-local declarations including generated helpers. Dependencies are restricted to the standard Lean foundations; no unfinished proofs or domain-specific axioms are admitted.

The package contains exact scalar, memory-partition and query experiments plus independent certificate checkers. The transition CLI imports the tested modular implementation. The Rust companion checks EIS metadata, role-label separation, digests and reported horizon arithmetic; it neither replays obstruction predicates nor authenticates host identities. Distinct labels alone do not establish statistical independence or evidential soundness. The ASCII identifier contract is checked in both languages. The asynchronous fixtures deliberately inject residuals from lag/skew/rewiring flags, including a positive age penalty when report contents agree. Their distribution shift is a property of that synthetic policy, not a derived transport law.

The cochain repair kernel minimizes an L1 residual over unrestricted potentials. Every successful certificate must reconstruct the repaired cochain from its potential, reproduce the residual/cost, annihilate the trans-

posed incidence with a dual vector of infinity norm at most one, and close the exact rational duality gap. Closedness alone is insufficient: the review's closed annulus has optimal repair cost 9. HiGHS supplies a numerical candidate; the exact checker can refuse it. Active repair replays the certificate and admits only an exact integer price, strictly positive for a nonzero repair, even with Python assertions disabled.

This cochain LP is separate from the companion's probability-marginal LP, which requires nonnegative normalized global distributions. There is no declared reduction and no general identification of contextual fraction with half the unweighted concatenated-context L1 distance. Two disjoint PR scenarios give contextual fraction 1 but distance 4. A support-gluing test ranges over families of supported local sections; restrictions of a single preselected global assignment already agree. Strong contextuality implies logical contextuality, which implies probabilistic contextuality. The review response links the original contextual-fraction definition. Source hashes and current censuses are supplied under `lean4/`; the review corrections are detailed in `docs/review_response_2026_10_23.md`. The live-memory and numerical verification files retain their earlier experiment dates and scopes. The paper and companion are independently buildable; neither imports its sibling's files. See the README for commands and `docs/embedded_research.md` for the claim-to-source map.

The pre-integration paper-000 source was version 1.0.0 with unreleased campaign additions. This edition is its explicit successor. A historical companion attribution to a separate version 1.4.0 recovery package is retained as historical custody, not treated as a source of the current proof estate. The current paired editions are this paper 1.2.3 and *Contextual Prediction and Self-Certification* 1.8.2.

8. Persistent memory in the live agent

The persistent experiment binds a declared task, model, action set, exact key/value encoding and update rule. Its categorical residual is zero exactly when all recorded observations sharing a key agree. Every recall and proposed update rechecks that residual, source hashes, exact source membership and the contract. Accepted facts enter the repository agent's user context before prompt binding. Rejected updates retain the previous archive; failed recalls prevent dispatch. SQLite transactions and a persisted database carry observations between processes.

The captured exact-use trial used four separate processes: an uninformed baseline returned UNKNOWN, the agent extracted a supplied synthetic observation, and two later processes used the stored value with different requested suffixes. An earlier arithmetic trial failed after correct recall. Thus persistent access and correct downstream reasoning remain different properties. Both outcomes and raw replies are retained, with independent offline replay of prompt bindings and source pins. This is bounded evidence from one model, not a general learning or intelligence benchmark. The connection is to the repository agent facade; it does not modify Codex's global context or model parameters.

The eight Lean statements in `LiveMemory` prove source-supported, consistent updates and recalls, preservation of old entries, refusal behavior and recall after an explicitly exact reload. SQLite crash recovery and the live model's use of context are tested operationally, not proved by these abstract statements. Source adequacy remains a premise. Implementation, commands and full limitations are in `docs/live_memory.md`; captured-trial replay uses `tools/replay_live_memory_evidence.py`.

The preceding numerical verification recorded all 712 quantum-library tests passing in release mode, with no failures, ignored cases or filtered cases. This includes the three prior numerical failures, all ten previously unfinished MUB tests, and three new admission-boundary regressions. Exact PSD of the supplied binary Choi matrix replaces a rounded eigenvalue-sign decision. Floating-point Hamiltonian evolution uses a separate tolerance-checked numerical propagator; exact admission remains strict, and numerical evolution retains norm drift. The rebuilt native budget host passes all 24 gateway-and-memory tests, including both previously blocked cases; five quantum integration binaries pass 57 tests. These are scoped checks of the earlier recorded source snapshot, not fresh native reruns in this edition. The earlier multi-crate audit remains historical, and these synthetic gateway tests add no new live model trial or universal implementation-refinement proof. The current source-bound build and test records are in `docs/numerical_verification.json`; `docs/native_gateway_verification.json` preserves the preceding partial run.

The companion Projection Obstruction extensions separate a measured residual from an error enclosure. PO-000 section 5.18 transfers uncertainty into onset and correction-window bounds; PO-033 section 3.5 accounts for integration defects and finite correction jumps; PO-034 section 4.3 gives density-state product-defect uncertainty at most (k+1) times the trace-norm state error. The preceding numerical-release report records their exact scalar witnesses and then-written continuous bounds. Later projection-paper mechanization is a separate release; these transferred observations do not supply automatic error certificates for recalled memories.

References

1. Wolpert, D. H. (2008). Physical limits of inference. Setup-distinguishability is essential.
2. Garrabrant et al. (2016). Logical induction.
3. Fallenstein, Taylor and Christiano (2015). Reflective oracles.
4. Shalizi and Crutchfield (2001). Computational mechanics.
5. Littman, Sutton and Singh (2001). Predictive representations of state. NeurIPS 14. Primary paper and locators in `docs/embedded_literature.json`.
6. Tishby, Pereira and Bialek (1999). The information bottleneck method.
7. Buscemi (2012). Comparison of quantum statistical models.
8. Scarlett and Cevher (2019). An introductory guide to Fano's inequality.
9. Chen, Papadimitriou and Peng (2022). The memory complexity of continual learning.
10. Brown, Bun and Smith (2022). Strong memory lower bounds for learning natural models. PMLR 178. Primary source recorded in the source catalog.
11. Legg and Hutter (2007). Universal intelligence.
12. Leike and Hutter (2015). Bad universal priors and notions of optimality.
13. Schmidhuber (2003/2005). Gödel machines.
14. Howard, Ramdas, McAuliffe and Sekhon (2021). Time-uniform confidence sequences.
15. Dwork et al. (2015). The reusable holdout.
16. Wu et al. (2026 revision). Statistical Gödel machines. Inspected version 2, September 17, 2026.